Master's Programme in Statistics and Machine Learning

732A99 Machine Learning

Computer Lab 1 Block 1

Lab Report

Jonas Danielsson
jonda228
Jonas Hautamäki Bengtsson
jonha849
Kerstin Arbring
kerar739



Division of Statistics and Machine Learning
Department of Computer and Information Science
Linköping University
November 13, 2026

1 Assignment 1

2 Assignment 2 - Linear regression and ridge regression

2.1 Data and ordinary linear regression

We predicted motor UPDRS from 16 voice characteristics in the Parkinson's dataset. The data were randomly divided into 3525 training observations (60%) and 2350 test observations (40%), using `set.seed(12345)`. Both the predictors and response were standardized using the *training-set* means and standard deviations, and the same transformation was applied to the test set. Consequently, the models were fitted without an intercept.

The ordinary least squares (OLS) model had training MSE 0.8785 and test MSE 0.9354 (on the standardized response scale). 8 coefficients were significant at the 5% level: Jitter(Abs), Shimmer, Shimmer:APQ5, Shimmer:APQ11, NHR, HNR, DFA, PPE.

2.2 Likelihood, Ridge objective and degrees of freedom

We assumed Normaly distrobuted errors and that `motor_UPDRS` is normally distributed around the value predicted by the regression model. Based on these assumption, we construct the log-likelihood used in Ridge regression.

$$Y_i \mid \mathbf{x}_i \sim N(\mathbf{x}_i^\top \theta, \sigma^2), \qquad \ell(\theta, \sigma) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \mathbf{x}_i^\top\theta)^2.$$

The implemented `Ridge` function evaluates the negative log-likelihood with an L2 penalty:

$$J(\theta, \sigma; \lambda) = -\ell(\theta, \sigma) + \lambda \sum_{j=1}^{16} \theta_j^2.$$

For each fixed λ, `RidgeOpt` minimizes this objective using `optim(method = "BFGS")`, jointly estimating the coefficients and σ. We optimized $\log\sigma$ instead of σ. This insure $\sigma > 0$, as we can not have a negative variance.

Following Lecture 1d (slides 17 and 22), the `DF` function uses the trace of the Ridge smoother matrix to measure effective model complexity:

$$\mathrm{df}(\lambda) = \mathrm{tr}\left[X(X^\top X + \lambda I)^{-1}X^\top\right].$$

2.3 Comparison of penalty strengths

The estimated optimal coefficients were used to predict responses by $\hat{\mathbf{y}} = X\hat{\theta}$. Table Table 1 compares the training and test MSE and the effective degrees of freedom for the three specified penalty strengths.

Table 1: OLS and Ridge results. MSE values are on the standardized response scale.

Model	Training MSE	Test MSE	Effective df	Sigma
OLS	0.8785	0.9354	16.0000	0.9373
Ridge (lambda = 1)	0.8786	0.9350	13.8607	0.9374
Ridge (lambda = 100)	0.8844	0.9323	9.9249	0.9404
Ridge (lambda = 1000)	0.9211	0.9540	5.6439	0.9598

Among the specified penalties, $\lambda = 100$ produced the lowest test MSE (0.9323), compared with 0.9354 for OLS. The lecture-defined effective degrees of freedom decreased from 13.86 at $\lambda = 1$ to 5.64 at $\lambda = 1000$. Thus, stronger regularization reduced the model's effective flexibility, although all 16 coefficients remained in the model. The test MSE indicates which of these penalty strengths gave the best predictive performance on the held-out data.

3 Assignment 3 - Logistic regression and Basis Function Expansion

Visualization of the data shows that there are values equal to zero in the measures, including plasma glucose which is one of the predictors of interest in this assignment. `Glucose = 0` is not a feasible value, but rather representing a missing value. All cases (n=5) with plasma glucose equal to zero were excluded from the data before the regression analysis.

3.1 Scatterplot diabetes vs age

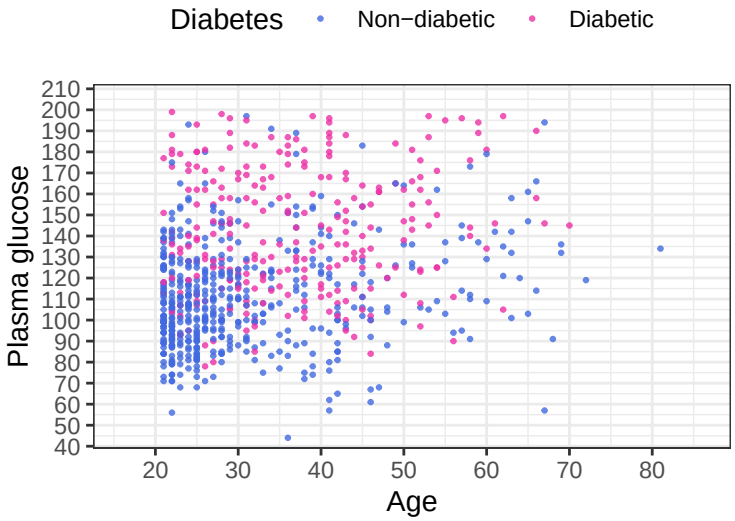


Figure 1: Plasma glucose levels in relation to age. Blue dots represent non-diabetic subjects, the pink dots diabetic subjects.

Plotting plasma glucose levels vs. age shows that the two response groups, *Non-diabetic* and *Diabetic*, show a substantial overlap. `Diabetes` would therefore not be easily predicted by *Age* and *Glucose* in a standard logistic regression model which would not be able to derive a straight line clearly separating the groups.

3.2 Logistic regression model

The vector form of the probailistic equation for a logistic regression model is

$$P(Y = 1 \mid x) = \text{sigmoid}(\theta^T x) \tag{1}$$

where the sigmoid function is

$$sigmoid(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \tag{2}$$

For *Diabetes* as the binary target/response (y), and *Glucose* and *Age* as features/predictors (x1, x2):

$$P(Y = 1 \mid x_1, x_2) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \cdot x_1 + \theta_2 \cdot x_2)}} \tag{3}$$

Training a logistic regression model on the (entire) provided data set, using glm() in R, results in the coefficients $\theta_0 = -6.2252$, $\theta_1 = 0.0384$, and $\theta_2 = 0.0231$, all with significant p values, and the corresponding probabilistic equation is then

$$P(Y = 1 \mid x_1, x_2) = \frac{1}{1 + e^{-(-6.2252 + 0.0384x_1 + 0.0231x_2)}} \tag{4}$$

The training misclassification error for this model (with r = 0.5) is 0.26, i.e approximately one out of four observations are misclassified.

3.3 Scatterplot colored by predicions

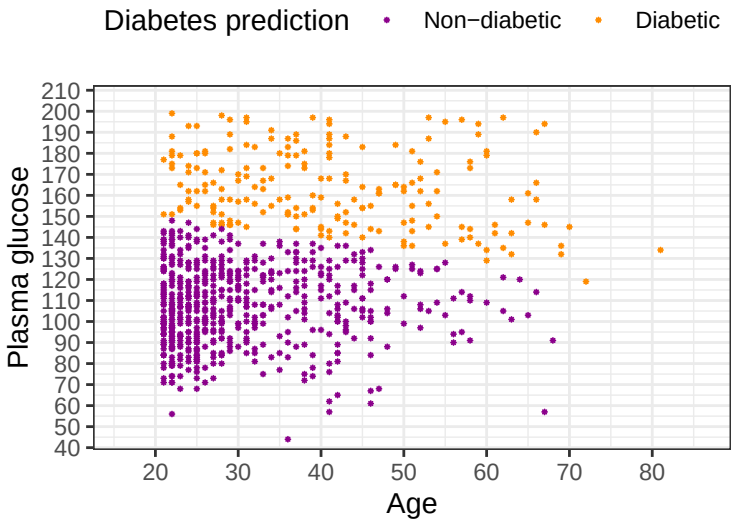


Figure 2: Plasma glucose levels in relation to age. Purple stars represent subjects predicted to be non-diabetic, the orange stars subjects predicted to be diabetic.

Comparing the predicted diabetes classifications to the actual diabetes status as shown in Figure 1 it looks like there will be some misclassifications, possibly more so for subjects in the lower, middle part of the scatter, where diabetic an non-diabetic subjects are overlapping, and for the non-diabetic elderly subjects.

3.4 Decision boundary equation

Setting the probabilistic equation in Equation 4 equal to `r` (here `= 0.5`) results in this approximate decision boundary equation, with the fitted coefficients and $x1 = Glucose$ and $x2 = Age$:

$$-6.2252 + 0.0384 \cdot Glucose + 0.0231 \cdot Age = 0 \tag{5}$$

Solving this for *Glucose* gives the approximate linear decision boundary line

$$Glucose = 161.97 - 0.6 \cdot Age \tag{6}$$

3.5 Scatterplot with decision boundary line for r = 0.5

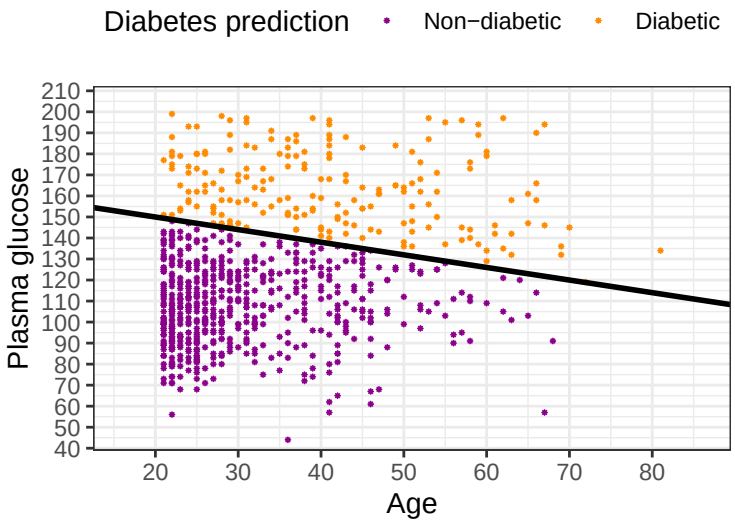


Figure 3: Predicted diabetes classifications with threshold r = 0.5. The added black line is the decision boundary line.

The boundary line means that there will likely be misclassifications both ways in the mixed "middle" group. The elderly non-diabetics will risk being misclassified as diabetic according to the negative slope of the linear boundary imposing increasingly harder restrictions on acceptable plasma glucose level for a non-diabetic subject.

3.6 Scatterplot with decision boundary line for r = 0.2 and r = 0.8

The equation for the linear decision boundary line for two predictors/features can generically be expressed as

$$x_1 = \frac{\log\left(\frac{r}{1-r}\right) - \theta_0}{\theta_1} - \frac{\theta_2}{\theta_1} x_2 \tag{7}$$

which means the intercept changes with r, the slope does not.

With r = 0.2 are more individuals predicted to be diabetic, with r increased to 0.8 instead fewer individuals. The training misclassification errors are 0.36 and 0.31 resepectively for the 0.2 and 0.8 cases, both higher than for the 0.5 cutoff case.

3.7 Basis function expansion

Fitting a new logistic regression model after adding new features

$$z_1 = x_1^4, \quad z_2 = x_1^3 x_2, \quad z_3 = x_1^2 x_2^2, \quad z_4 = x_1 x_2^3, \quad z_5 = x_2^4$$

results in a much more complex model but a training misclassification error (with r = 0.5) of 0.24 which is slightly better than the linear model. The decision boundary is now not linear but curvilinear, changing the appearance of the Glucose vs Age scatterplot (see Figure 5). The most

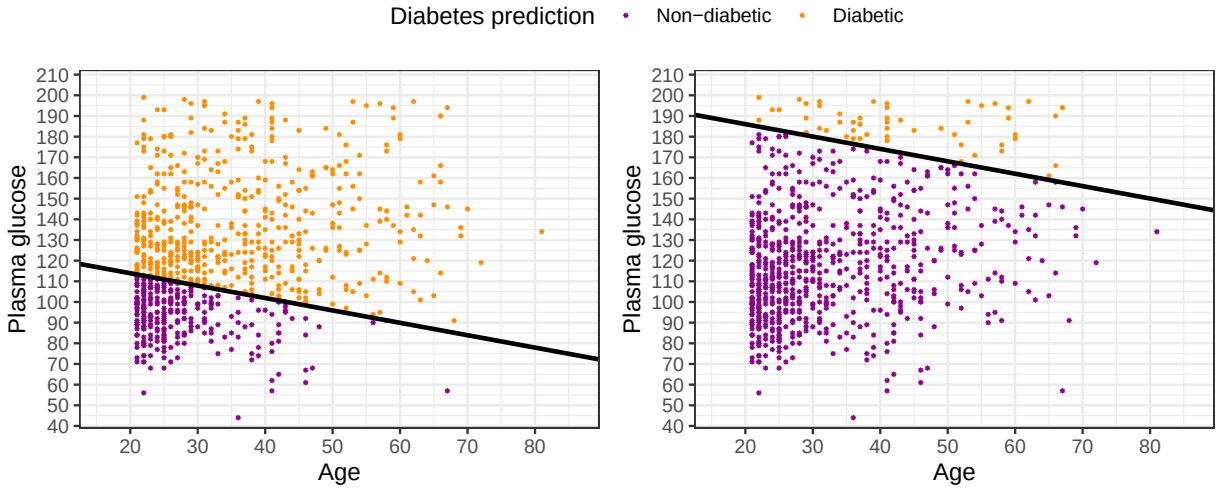


Figure 4: Predicted diabetes classifications for thresholds r = 0.2 (left) and r = 0.8 (right). Black lines indicate the decision boundaries.

obvious difference from the previous model is that older subjects are "allowed" to have higher plasma glucose levels without being predicted to be diabetic, which is a pattern that can be discerned also in the actual data.

The *theta3* to *theta7* coefficients (for the *z1* to *z5* terms) are very small (magnitude $10^{-7} or less), and individually non-significant (p values above 0.6), but they are multiplied with numerically high values for *Glucose* (raised to up to 4) in the polynomial, and the AIC reduction suggests an improved overall fit for the model.

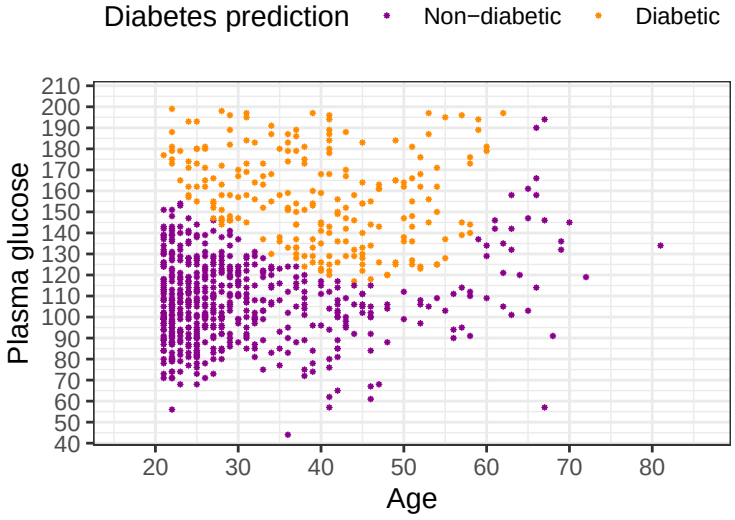


Figure 5: Predicted diabetes classifications for the extended logistic regression model with polynomial basis function expansion.

4 Theory

A R code

A.1 Assignment 1

A.2 Assignment 2

```
# Lab 1, Assignment 2 - Linear regression and ridge regression
#
# Sourced by sections/lab1_assignment2.qmd. Paths are relative to Lab1/.
# Prefix objects with a2_ so they don't clash with other assignments'
# scripts when everything is sourced into the combined report.

# Load packages ----------------------------------------------------------------
require(tidyverse)
require(caret)

# read data --------------------------------------------------------------------

a2_data <- read.csv("data/parkinsons.csv", check.names = FALSE)

# 1. Train/test split (60/40) and scaling --------------------------------------

# keep only the target (motor_UPDRS) and the 16 predictors;
# drop variables subject#, age, sex, test_time and total_UPDRS
a2_voice_vars <- c("Jitter(%)", "Jitter(Abs)", "Jitter:RAP", "Jitter:PPQ5",
                   "Jitter:DDP", "Shimmer", "Shimmer(dB)", "Shimmer:APQ3",
                   "Shimmer:APQ5", "Shimmer:APQ11", "Shimmer:DDA",
                   "NHR", "HNR", "RPDE", "DFA", "PPE")
a2_data <- a2_data[, c("motor_UPDRS", a2_voice_vars)]

# partition 60/40 (code from: Lecture 1a, slide 34)
n <- dim(a2_data)[1]
set.seed(12345)
id <- sample(1:n, floor(n * 0.6))
a2_train_raw <- a2_data[id, ]
a2_test_raw  <- a2_data[-id, ]

# scale with the TRAINING means and sds, also for the test data, so that no
# information from the test set leaks into the model.
#
# motor_UPDRS is scaled too (centred at 0), which is why no intercept is needed.
a2_means <- colMeans(a2_train_raw)
a2_sds   <- map_dbl(a2_train_raw, sd)

a2_train <- as.data.frame(scale(a2_train_raw, center = a2_means, scale = a2_sds))
a2_test  <- as.data.frame(scale(a2_test_raw,  center = a2_means, scale = a2_sds))
```

```
# 2. Linear regression -------------------------------------------------------

# linear regression on all 16 voice variables
# without intercept as it is scaled and screws with ridge penalty
a2_lm <- lm(motor_UPDRS ~ . - 1, data = a2_train)
a2_lm_summary <- summary(a2_lm)

# coefficient table
a2_lm_coefs <- as.data.frame(a2_lm_summary$coefficients)

# variables that are significant at the 5% level
a2_lm_signif <- rownames(a2_lm_coefs)[a2_lm_coefs[, "Pr(>|t|)"] < 0.05]

# MSE = mean squared diff between observed and predicted values
## Created MSE function (There DO exist package functions for MSE)
a2_mse <- function(y, y_hat) mean((y - y_hat)^2)

## Calculating MSE
a2_lm_train_mse <- a2_mse(a2_train$motor_UPDRS,
                          predict(a2_lm, newdata = a2_train))
a2_lm_test_mse  <- a2_mse(a2_test$motor_UPDRS,
                          predict(a2_lm, newdata = a2_test))


# 3. Functions (base R only, no external packages) ---------------------------

# training data as matrix/vector, used by all functions in step 3
a2_X_train <- as.matrix(a2_train[, a2_voice_vars])   # n x 16 feature matrix
a2_y_train <- a2_train$motor_UPDRS                   # length-n target vector

# 3a. log P(T | theta, sigma) for the training data (T)
## This assumes that the error is Normal
a2_loglikelihood  <- function(theta, sigma) {
  n <- length(a2_y_train)
  residuals <- a2_y_train - a2_X_train %*% theta
  # log liklihood for normal distrobution
  -n / 2 * log(2 * pi * sigma^2) - sum(residuals^2) / (2 * sigma^2)
}

# 3b. minus log-likelihood + lambda * ||theta||^2  (obligatory oral defence)
a2_ridge <- function(theta, sigma, lambda) {
  neg_loglik <- -a2_loglikelihood(theta,sigma)
  penalty <- lambda * sum(theta^2)
  neg_loglik + penalty
}

# 3c. optimal theta and sigma for a given lambda, using optim(method = "BFGS")
```

```r
#
# optim() minimises a function of ONE parameter vector (in this case 'par'),
# so theta (16 values) and sigma are packed together:
# AKA par = c(theta, log(sigma)). One vector.
a2_ridge_opt <- function(lambda) {
  # numeber of paramaters
  p <- ncol(a2_X_train)

  # objective -- function to use in optim (a function to be minimized)
  # Unpacks par and evaluate the ridge function from 3b
  objective <- function(par) {
    theta <- par[1:p]
    sigma <- exp(par[p + 1])
    # returns minus log-likelihood of the training data + ridge penalty
    # (the value optim minimises)
    a2_ridge(theta, sigma, lambda)
  }

  # starting values: theta = 0, sigma = 1 (log(1) = 0)
  start <- c(rep(0, p), 0)

  fit <- optim(par = start, fn = objective, method = "BFGS")

  theta_hat <- fit$par[1:p]
  names(theta_hat) <- colnames(a2_X_train)

  list(theta       = theta_hat,
       sigma       = exp(fit$par[p + 1]),
       value       = fit$value,        # minimised ridge objective
       convergence = fit$convergence)  # 0 = optim converged
}

# 3d. degrees of freedom of the ridge model for a given lambda
a2_df <- function(lambda) {
  X <- a2_X_train
  p <- ncol(X)
  # Ridge hat matrix (smoother matrix) (Lecture 1d, slide 22)
  P <- X %*% solve(t(X) %*% X + lambda * diag(p)) %*% t(X)
  # df_eff = tr(P) trace of a matrix: the som of it's diagonal entries
  # (Lecture 1d, slide 17)
  sum(diag(P))
}


# 4. Ridge for lambda = 1, 100, 1000 ------------------------------------------

a2_lambdas <- c(1, 100, 1000)
```

```r
# test data as matrix, same column order as a2_X_train
a2_X_test <- as.matrix(a2_test[, a2_voice_vars])
a2_y_test <- a2_test$motor_UPDRS

# optimal theta and sigma for each lambda (using ridge_opt (from 3c) per lambda)
a2_ridge_fits <- lapply(a2_lambdas, a2_ridge_opt)
names(a2_ridge_fits) <- paste0("lambda_", a2_lambdas)

# predictions are X %*% theta (optimal theta from ridge_opt) (no intercept),
# then MSE, df and sigma per lambda
a2_ridge_results <- data.frame(
  lambda      = a2_lambdas,
  train_mse   = sapply(a2_ridge_fits, \(f) a2_mse(a2_y_train, a2_X_train %*% f$theta)),
  test_mse    = sapply(a2_ridge_fits, \(f) a2_mse(a2_y_test,  a2_X_test  %*% f$theta)),
  df          = sapply(a2_lambdas, a2_df),
  sigma       = sapply(a2_ridge_fits, \(f) f$sigma),
  convergence = sapply(a2_ridge_fits, \(f) f$convergence),  # 0 = converged
  row.names   = NULL
)

# coefficients side by side -- linear regression and ridge per lambda
a2_ridge_coefs <- cbind(lm = coef(a2_lm),
                        sapply(a2_ridge_fits, \(f) f$theta))

# Values used in report ---------------------------------------------

## fixing name formating (removing backtick formating from factors)
a2_significant_names <- gsub("`", "", a2_lm_signif, fixed = TRUE)

## significant coef print
a2_significance_sentence <- paste0(
  length(a2_significant_names),
  " coefficients were significant at the 5% level: ",
  paste(a2_significant_names, collapse = ", "), "."
)

# print values
a2_sigma_mle <- sqrt(mean(residuals(a2_lm)^2))
a2_best_idx <- which.min(a2_ridge_results$test_mse)
a2_best_lambda <- a2_ridge_results$lambda[a2_best_idx]
a2_best_test_mse <- a2_ridge_results$test_mse[a2_best_idx]

# Final comparision table
a2_comparison <- rbind(
  data.frame(
    Model = "OLS",
    `Training MSE` = a2_lm_train_mse,
```

```
    `Test MSE` = a2_lm_test_mse,
    `Effective df` = ncol(a2_X_train),
    `Sigma` = a2_sigma_mle,
    check.names = FALSE
  ),
  data.frame(
    Model = paste0("Ridge (lambda = ", a2_ridge_results$lambda, ")"),
    `Training MSE` = a2_ridge_results$train_mse,
    `Test MSE` = a2_ridge_results$test_mse,
    `Effective df` = a2_ridge_results$df,
    `Sigma` = a2_ridge_results$sigma,
    check.names = FALSE
  )
)
```

A.3 Assignment 3

A.3.1 Assignment 3 — Functions

```
################################################################################
# function definitions

################################################################################
#' A3_classifications_and_plots
#' Adds predictions based on provided predicted probabilities
#' and decision threshold, and creates a plot colored by diabetes prediction
#' and a plot colored by the predicion being correct or not,
#' the latter is extra so not presented in report.
#'
#' @param data_pred_prob A tibble with data and predicted probabilities
#' @param r Classification threshold
#' @param glucose_range Numerical vector with min and max of Glucose values
#' @param age_range Numerical vector with min and max of Age values
#'
#' @returns A list of:
#' data_with_predictions
#'    = data where Diabetes_prediction and Correct_prediction
#'    columns (both logical) are added
#' confusion_matrix
#'    = confusion matrix for Actual vs Predicted diabetes
#' correct_proportion
#'    = Proportion of correctly classified observations
#' training_misclass_error
#'    = Proportion of misclassified observations
#' pred_plot
#'    = Plot of Glucose vs Age, colored by Diabetes_predicion
```

```
#' class_plot
#'    = Plot of Glucose vs Age, colored by Correct_pred
#'
A3_classifications_and_plots <-
  function(data_pred_prob, r, glucose_range, age_range){

  # adding prediction classifications to the data tibble
  data_with_predictions <-
    data_pred_prob |>
    mutate(Diabetes_prediction = ifelse(Predicted_prob > r, TRUE, FALSE),
           Correct_pred = (Diabetes == Diabetes_prediction))

  # confusion matrix
  logistic_confusion <-
    table(Actual = data_with_predictions$Diabetes,
          Predicted = data_with_predictions$Diabetes_prediction)

  # proportion correctly classified observations
  correct_proportion <-
    sum(diag(logistic_confusion))/sum(logistic_confusion)

  # training misclassification error
  # = proportion of misclassified observations
  training_misclass_error <-
    1 - correct_proportion

  # scatterplot colored by predictions
  pred_plot <-
    data_with_predictions |>
    ggplot() +
    aes(x = Age, y = Glucose, color = Diabetes_prediction) +
    geom_point(size = 0.3, shape = 8) +
    theme_bw() +
    theme_bw() +
    theme(legend.position = "top") +
    scale_colour_manual(labels = c("Non-diabetic", "Diabetic"),
                        values = c("darkmagenta", "darkorange")) +
    scale_x_continuous(
      limits = age_range + c(-5, +5),
      breaks = scales::breaks_width(10)) +
    scale_y_continuous(
      limits = glucose_range + c(-5, +5),
      breaks = scales::breaks_width(10),
      expand = expansion(mult = c(0, 0.05))
    ) +
    labs(y = "Plasma glucose", color = "Diabetes prediction")
```

```
  # scatterplot colored by correct or misclassfication
  class_plot <-
    data_with_predictions |>
    ggplot() +
    aes(x = Age, y = Glucose, color = Correct_pred, shape = Correct_pred) +
    geom_point(size = 0.3) +
    theme_bw() +
    theme(legend.position = "top") +
    scale_colour_manual(
      name = "Classification result",
      values = c(`TRUE` = "green", `FALSE` = "red"),
      labels = c(`TRUE` = "Correct", `FALSE` = "Misclassification")
    ) +
    scale_shape_manual(
      name = "Classification result",
      values = c(`TRUE` = 18, `FALSE` = 4),
      labels = c(`TRUE` = "Correct", `FALSE` = "Misclassification")
    ) +
    scale_x_continuous(
      limits = age_range + c(-5, +5),
      breaks = scales::breaks_width(10)) +
    scale_y_continuous(
      limits = glucose_range + c(-5, +5),
      breaks = scales::breaks_width(10),
      expand = expansion(mult = c(0, 0.05))
    )

  return(list(data_with_predictions = data_with_predictions,
              confusion_matrix = logistic_confusion,
              correct_proportion = correct_proportion,
              training_misclass_error = training_misclass_error,
              pred_plot = pred_plot,
              class_plot = class_plot))
}

################################################################################
#' A3_calculate_linear_boundary
#' Calculating a linear boundary intercept and slope
#'
#' @param theta0 Fitted intercept
#' @param theta1 Fitted coefficient for Glucose
#' @param theta2 Fitted coefficient for Age
#'
#' @returns A list with boundary intercept and slope
#'
A3_calculate_linear_boundary <- function(theta0, theta1, theta2, r){
```

```
  # decision boundary line intercept and slope
  boundary_intercept <-  (log(r/(1-r))-theta0)/theta1
  boundary_slope <-  -theta2/theta1

  return(list(boundary_intercept = boundary_intercept,
              boundary_slope =  boundary_slope))
}

################################################################################
#' A3_add_linear_boundary
#' Adds a decision boundary line to the provided plots
#'
#' @param results_list List returned by
#' A3_classifications_and_plots
#'
#' @returns A list of two plots,
#' one colored by diabetes prediction, and
#' one colored by the predicion being correct or not
#'
A3_add_linear_boundary <-  function(results_list, r ){
  # calculate intercept and slope for the straight line
  boundary <-
    A3_calculate_linear_boundary(A3_theta0, A3_theta1, A3_theta2, r)

  # add line to prediction plot
  pred_plot_with_line <-
    results_list[["pred_plot"]] +
    geom_abline(
      intercept = boundary[["boundary_intercept"]],
      slope = boundary[["boundary_slope"]],
      color = "black",
      linewidth = 1
    )

  # add line to classification correctness plot
  class_plot_with_line <-
    results_list[["class_plot"]] +
    geom_abline(
      intercept = boundary[["boundary_intercept"]],
      slope = boundary[["boundary_slope"]],
      color = "black",
      linewidth = 1
    )
  return(list(pred_plot_with_line=pred_plot_with_line,
              class_plot_with_line=class_plot_with_line))
}
```

```
################################################################################
```

A.3.2 Assignment 3 — Main script

```
library(tidyverse)

# Data in pima-indians-diabetes.csv:
# 1. Number of times pregnant.
# 2. Plasma glucose concentration at 2 hours in an oral glucose tolerance test.
# 3. Diastolic blood pressure (mm Hg).
# 4. Triceps skinfold thickness (mm).
# 5. 2-Hour serum insulin (mu U/ml).
# 6. Body mass index (weight in kg/(height in m)^2).
# 7. Diabetes pedigree function.
# 8. Age (years).
# 9. Diabetes (0=no or 1=yes).

# import user-definied functions
source("R/lab1_assignment3_functions.R")

# read data
A3_pimadata <- read.csv("data/pima-indians-diabetes.csv")

# any NA:s?
if (anyNA(A3_pimadata)) {
  message("NA:s in table")
}

# create tibble with meaningful column names
A3_pimatibble_raw <-
  A3_pimadata |>
  tibble() |>
  magrittr::set_colnames(c("Parity", "Glucose", "DBP", "Skinfold",
                           "Insulin","BMI", "Pedigree", "Age", "Diabetes")) |>
  mutate(Diabetes = as.logical(Diabetes)) |>
  mutate(ID = row_number(), .before = Parity) |>
  select(ID, Age, Pedigree, Parity, BMI, Skinfold,
         DBP, Insulin, Glucose, Diabetes)

# data check
  A3_pimatibble_raw |>
  pivot_longer(-c(ID,Diabetes),
               names_to = "Measure",
               values_to = "Value") |>
  ggplot() +
  aes(x=Value, fill = Diabetes) +
```

```
  geom_histogram(bins = 30) +
  scale_fill_manual(labels = c("Non-diabetic", "Diabetic"),
                    values = c("royalblue", "maroon2")) +
  facet_wrap(~ Measure, scales = "free", nrow = 2) +
  theme_bw() +
  theme(legend.position = "top")

# checking exact range for Age, Glucose, and Diabetes
A3_pimatibble_raw |>
  select(c(Age, Glucose, Diabetes)) |>
  map(\(x) range(x))

# removing cases with Glucose = 0
A3_pimatibble <-
  A3_pimatibble_raw |>
  filter(Glucose != 0)

# saving adjusted ranges
A3_minmaxlist <-
  A3_pimatibble |>
  select(Age, Glucose) |>
  map(\(x) tibble(min = min(x), max = max(x)))

A3_Age_range <- unlist(A3_minmaxlist[["Age"]])
A3_Glucose_range <- unlist(A3_minmaxlist[["Glucose"]], use.names = FALSE)

# scatterplot Glucose vs Age

A3_Glucose_vs_Age_scatter <-
  A3_pimatibble |>
  ggplot() +
  aes(x = Age, y = Glucose, color = Diabetes) +
  geom_point(size = 0.5, alpha = 0.8) +
  theme_bw() +
  theme(legend.position = "top") +
  scale_colour_manual(labels = c("Non-diabetic", "Diabetic"),
                      values = c("royalblue", "maroon2")) +
  scale_x_continuous(
    limits = A3_Age_range + c(-5, +5),
    breaks = scales::breaks_width(width=10)) + # equally spaced breaks
  scale_y_continuous(
    limits = A3_Glucose_range + c(-5, +5),
    breaks = scales::breaks_width(10),
    expand = expansion(mult = c(0, 0.05))
  ) +
  labs(y = "Plasma glucose")
```

```r
# training a logistic regression model

# Diabetes ~ Glucose + Age in a logistic regression model
# using glm()= with family = "binomial" since target has two levels

A3_logistic_model <-
  glm(Diabetes ~ Glucose + Age, data = A3_pimatibble, family = "binomial")

summary(A3_logistic_model)

# coefficients (= coefficients in the probabilistic equation)

A3_logistic_coef <- coef(A3_logistic_model)

A3_theta0 <- A3_logistic_coef["(Intercept)"]
A3_theta1 <- A3_logistic_coef["Glucose"]
A3_theta2 <- A3_logistic_coef["Age"]

# adding predicted probabilities to data
A3_pimatibble_pred_probs <-
  A3_pimatibble |>
  mutate(Predicted_prob = predict(A3_logistic_model, type = "response"))

# adding predicted diabetess classification
# and comparison with actual diabetes class
# and plot visualizing the results
A3_results_r_05 <-
  A3_classifications_and_plots(A3_pimatibble_pred_probs,
                               r = 0.5, A3_Glucose_range, A3_Age_range)

A3_results_r_02 <-
  A3_classifications_and_plots(A3_pimatibble_pred_probs,
                               r = 0.2, A3_Glucose_range, A3_Age_range)

A3_results_r_08 <-
  A3_classifications_and_plots(A3_pimatibble_pred_probs,
                           r = 0.8, A3_Glucose_range, A3_Age_range)

# adding decision boundary lines to Glucose vs Age plots
A3_boundary_05_plots <-
  A3_add_linear_boundary(A3_results_r_05, r = 0.5)

A3_boundary_02_plots <-
  A3_add_linear_boundary(A3_results_r_02, r = 0.2)

A3_boundary_08_plots <-
  A3_add_linear_boundary(A3_results_r_08, r = 0.8)
```

```
# basis function expansion
# sdding new features
A3_pimatibble_extended <-
  A3_pimatibble |>
  mutate(z1 = Glucose^4,
         z2 = Glucose^3*Age,
         z3 = Glucose^2*Age^2,
         z4 = Glucose*Age^3,
         z5 = Age^4)

A3_logistic_model_extended <-
  glm(Diabetes ~ Glucose + Age + z1 + z2 + z3 + z4 + z5,
      data = A3_pimatibble_extended, family = "binomial")

summary(A3_logistic_model_extended)

# coefficients (= coefficients in the probabilistic equation)

A3_logistic_extended_coef <- coef(A3_logistic_model_extended)

A3_theta0_ext <- A3_logistic_extended_coef["(Intercept)"]
A3_theta1_ext <- A3_logistic_extended_coef["Glucose"]
A3_theta2_ext <- A3_logistic_extended_coef["Age"]
A3_theta3_ext <- A3_logistic_extended_coef["z1"]
A3_theta4_ext <- A3_logistic_extended_coef["z2"]
A3_theta5_ext <- A3_logistic_extended_coef["z3"]
A3_theta6_ext <- A3_logistic_extended_coef["z4"]
A3_theta7_ext <- A3_logistic_extended_coef["z5"]


# adding predicted probabilities to data
A3_pimatibble_extended_pred_probs <-
  A3_pimatibble_extended |>
  mutate(Predicted_prob = predict(A3_logistic_model_extended, type = "response"))



A3_results_extended <-
  A3_classifications_and_plots(A3_pimatibble_extended_pred_probs,
                               r = 0.5,
                               A3_Glucose_range, A3_Age_range)
```